TRELLIS.2: licences and their costs, weights, GPU, and everything before deployment

TRELLIS.2 is the Microsoft model that turns a machine's 360 video into its 3D model, run as a GPU job in Flex's AWS. This document gathers everything that has to be settled or done before it can make models in production, and what the licences and access to the weights cost:

1. **Section 1:** where each item stands.
2. **Section 2:** licences.
3. **Section 3:** access to the model weights.
4. **Section 4:** the GPU to run on in Sydney.
5. **Section 5:** the steps before the first deployment.
6. **Section 6:** licence and access costs.

Running TRELLIS.2 on AWS, and what that costs, is in `docs/deployment-guide.md` (5.13).

The detailed commands for each step are in `docs/deployment-guide.md` (Part 5) and `docs/runbooks/trellis2-worker.md`; this document refers to them rather than repeating them.

What is built. The worker and its image (`gpu/trellis2/`), the GPU environment (`infra/lib/gpu-stack.ts`), the backend's side, the reviewer's screens, the alarms and the image pipeline. **What has not happened yet:** none of it has run on a real GPU or in a real AWS account, and the image has never been compiled. Every GPU timing is therefore an estimate until the first golden test and the first real builds are measured.

Checked without a GPU. The path from the phone to the walkthrough is wired end to end, and each step has tests that run without a GPU or AWS:

- **The 360 video** goes from the iPhone app to the captures bucket in signed parts, under the machine's own folder, which is the folder the GPU job is allowed to read.
- **A build starts only when a reviewer asks** in the editor (Build the model, Build another). An upload starts nothing, and with nothing to do there are no GPU hosts at all.
- **The job** is submitted to Batch with the video's S3 path, the try's own output folder and the scanned size. The backend's role may submit it and look it up, and a try is never sent twice.
- **The outcome** comes back through EventBridge to the encrypted `gpu-jobs` queue, whose queue policy and key policy both let EventBridge in. The worker then makes the model ready (levels of detail, KTX2), puts it in front of the reviewer, and settles any job whose event went missing.
- **Approved models** reach every walkthrough as presigned links to the models bucket, which the browser may read (CORS, CSP).

What only a real GPU can show: the image compiling, TRELLIS.2 running on the A10G, its timings, and how real recordings come out. Those are items 6 to 8 below, after items 2 to 5.

---

1. Where each item stands

#	Item	State	Who	Blocks
1	**The GPU type:** `g5.2xlarge` (A10G), in place of `g6e.xlarge`, which Sydney does not offer (section 4)	Done, in the code	Tejesh	—
2	**nvdiffrast licence (O-07):** non-commercial use only (section 2)	Open	Legal, management	Models for customers (prod)
3	**DINOv3 access** on Hugging Face (section 3)	Not requested	Someone who can bind the company	Copying the weights
4	**GPU quota** in dev, staging and prod (section 5)	Not requested	Flex's AWS team	Any GPU job
5	**Real AWS account numbers** in `infra/cdk.json`	Placeholders	Thomas, Flex's AWS team	Any deployment
6	**The first image build**, never compiled yet	Not started	Tejesh	The first GPU job
7	**Weights copied** into each environment's bucket	Not started	Tejesh	The first GPU job
8	**The golden reference** recorded from the first image	Not started	Tejesh	Testing new images automatically

Items 2 to 4 wait on decisions, people or AWS, so start them first, side by side.

---

2. Licences

Everything TRELLIS.2 uses

Part	What it is	Licence	Commercial use
TRELLIS.2 code (`microsoft/TRELLIS.2`)	The model's code	MIT	Yes
TRELLIS.2-4B weights	The model	MIT	Yes
TRELLIS-image-large (one decoder)	A weight file TRELLIS.2 reuses	MIT	Yes
DINOv3 (`facebook/dinov3-vitl16-pretrain-lvd1689m`)	The image encoder	DINOv3 License	Yes, once accepted (below)
BiRefNet (`ZhengPeng7/BiRefNet`)	Background removal, used **in place of RMBG-2.0**, which is non-commercial (CC BY-NC 4.0) and is not used	MIT	Yes
CuMesh, its cubvh submodule, FlexGEMM, utils3d	Mesh and sparse-convolution libraries	MIT	Yes

Part	What it is	Licence	Commercial use
o-voxel's Eigen	Linear algebra, header-only, unmodified	MPL 2.0	Yes
flash-attn	Attention kernels	BSD 3-Clause	Yes
PyTorch; the CUDA runtime in the base image	Framework and GPU runtime	BSD-style; NVIDIA's CUDA licence	Yes
nvdiffrast (`NVlabs/nvdiffrast`)	Texture baking in the GLB export	NVIDIA Source Code License, "1-Way Commercial"	**No**: non-commercial research or evaluation only (section 3.3)

`gpu/trellis2/README.md` ("Licence-safe configuration") records how the configuration avoids RMBG-2.0, and the pins of each part.

DINOv3: accept the licence

Read at the pinned revision, the DINOv3 License says:

- commercial use is allowed;
- it must be accepted by someone who can bind the company, which is what the gated access request on Hugging Face does;
- research publications must acknowledge DINOv3;
- no "Built with DINOv3" credit is required.

The weights are not redistributed: they run in our GPU job, and the licence file is kept beside them.

Cost: none.

nvdiffrast: the one blocker

TRELLIS.2's GLB export uses nvdiffrast to bake textures. Its licence allows use only for non-commercial research or evaluation, other than by NVIDIA. Making models for paying customers is commercial use.

Option	What it takes	Cost
1. Legal decides our use is allowed as it stands	Legal reads the licence	Legal's time
2. A commercial licence from NVIDIA	A request through NVIDIA's research licensing enquiries	Set by NVIDIA, on request; not published
3. Replace nvdiffrast in the texture bake	The 3D team rewrites the worker's export step without nvdiffrast, with its own task and tests	Engineering time; no licence fee

Meanwhile:

- Ask Legal, in the same request, whether internal evaluation on dev, with our own test recordings and not customer data, counts as "evaluation".
- Prod waits for the decision.

Done when the decision is recorded in O-07, and if it is option 3, the replacement is built.

3. Access to the weights

The weights (32 files, about 18 GB) come from Hugging Face. They are locked by revision and SHA-256 in `gpu/trellis2/weights.lock.json`. Today 29 files are locked; DINOv3's three are under `pending`, because they cannot be read without access.

The Hugging Face token never goes into AWS. It is used twice, on a machine you control:

- once to lock DINOv3's checksums;
- once per environment to copy the weights into that environment's own S3 bucket.

GPU jobs read the weights only from that bucket, with Hugging Face switched off. Keep the token in the company's password manager, never in the repository, `cdk.json`, Secrets Manager or the image.

Steps (details in `deployment-guide.md`, 5.2, 5.4, 5.5):

1. **Request access.** A company Hugging Face account requests DINOv3 access at https://huggingface.co/facebook/dinov3-vitl16-pretrain-lvd1689m, accepting the licence (section 2). Meta approves by hand.
2. **Make a token.** Create a Read token on that account, or a fine-grained one with "Read access to contents of all public gated repos you can access". Check it, with the command in the guide.
3. **Lock DINOv3's checksums:** `python lock_weights.py` in `gpu/trellis2`, with the token. Run the tests, then commit the lock. The push starts an image build, because the lock goes into the image.
4. **Copy the weights into each environment's bucket**, dev first: `python3 publish_weights.py s3://<models-bucket>/gpu/trellis2/weights/ --work <folder>`, with the token. It checks every file against the lock, uploads them, and writes `READY` last.
    - It needs about 20 GB of disk, and permission to write that bucket and use its KMS key. The policy is in the guide.
    - An EC2 instance in that account in Sydney is quickest.
5. **Afterwards:** delete the work folder, and stop the instance. The token can be revoked until it is next needed (a new lock, or a new environment).

Cost: Hugging Face and DINOv3 are free (section 6).

---

4. The GPU in Sydney

What changed

The code, the guide and the earlier plan first chose `g6e.xlarge` (one NVIDIA L40S, 48 GB) in Sydney. **AWS does not offer G6e in Sydney:**

- AWS's on-demand price list for Sydney has no `g6e` size at all;
- AWS lists G6e only in US East, US West (Oregon), Tokyo, Seoul, the UAE, Frankfurt, Spain and Stockholm.

Customer data must stay in Sydney (`docs/residency.md`), so the work cannot move to one of those regions. The GPU type has to change.

The options in Sydney

On-demand list prices, USD an hour:

Instance	GPU (memory)	vCPU, RAM	Local disk	Price	Runs today's image?
`g5.2xlarge`	NVIDIA A10G (24 GB)	8, 32 GiB	450 GB NVMe	**$1.5758**	Yes: the image is built for Ampere (8.6)
`g6.2xlarge`	NVIDIA L4 (24 GB)	8, 32 GiB	450 GB NVMe	**$1.2711**	Yes: Ada (8.9), like the L40S
`p5.4xlarge`	NVIDIA H100 (80 GB)	16, 256 GiB	3.84 TB NVMe	$8.944	No: needs the image rebuilt for Hopper (9.0)
`g5.xlarge`, `g6.xlarge`	24 GB	4, 16 GiB	250 GB	$1.31, $1.05	Too little RAM: the job needs 28 GiB

Memory. 24 GB is TRELLIS.2's stated minimum; the RunPod kit recommended 48 GB. At the worker's defaults (resolution 1024, a 2048 texture), 24 GB is expected to fit, but with little room. The highest resolution (1536) may run out of memory.

Decided: `g5.2xlarge`. The A10G has twice the memory bandwidth of the L4, which usually makes this kind of model faster, so it should cost the same or less per model despite the higher hourly price. The backend asks for no resolution, so every job runs at the default 1024, which is expected to fit in 24 GB.

Changed in the code:

- `infra/lib/gpu-stack.ts`: the instance type, and 8 vCPUs a host and a job; the 28 GiB job memory is unchanged and fits the 32 GiB host;
- its tests (`infra/test/gpu.test.ts`);
- the GPU quota: 8 vCPUs a host, not 4 (`docs/runbooks/service-quotas.md`);
- the guide, the diagram and the runbooks.

If the first real builds disagree:

- **The L4 is cheaper per model:** change `INSTANCE_TYPE` in `infra/lib/gpu-stack.ts` to `g6.2xlarge`. Nothing else changes: same vCPUs and memory, and the image already runs on it.
- **24 GB runs out:** move to `p5.4xlarge`. That costs about six times as much an hour, needs 16 vCPUs a host and a bigger quota, and the image rebuilt for Hopper (9.0).

---

5. Everything before the first deployment, in order

Each step names its owner, and where the guide (`docs/deployment-guide.md`) has the detail.

Step	What	Who	Guide
1	**Done:** the GPU type is `g5.2xlarge` (section 4), in the GPU stack	Tejesh	—
2	Ask Legal about nvdiffrast (section 2)	Tejesh, Legal	5.1
3	Request DINOv3 access (section 3)	The company	5.2
4	Request the GPU quota, "Running On-Demand G and VT instances" (`L-DB2E81BA`), in each environment account. Hosts have 8 vCPUs: ask for at least 16 in dev and staging and 32 in prod, to leave room.	Flex's AWS team	5.3
5	Put the real account numbers in `cdk.json`; bootstrap the accounts; deploy the pipelines and the environments, without a GPU yet	Thomas	2.1 to 2.6
6	Lock DINOv3's weights and push	Tejesh	5.4
7	Copy the weights into each environment's bucket	Tejesh	5.5
8	Build the first image through the image pipeline, and fix the build if the first compile fails	Tejesh	5.6
9	Turn on the GPU environment in dev with that image	Tejesh	5.7
10	The first golden test on dev; look at the model; record the reference	Tejesh	5.8
11	A real machine, end to end, on dev, with our own recordings	Tejesh, a reviewer	5.9
12	Staging: weights, image, golden test	Tejesh, Thomas	5.10
13	Prod, once nvdiffrast is settled	Thomas, an approver	5.11

---

6. Licence and access costs

What the licences and getting the weights cost. Running TRELLIS.2 on AWS is a separate matter, in `docs/deployment-guide.md`, 5.13.

Item	Licence fee	Ongoing (a subscription, or per model)	What it asks of us
TRELLIS.2 code and TRELLIS.2-4B weights (Microsoft)	None	None	MIT: keep Microsoft's copyright and licence notice with the code. The image does.
TRELLIS-image-large decoder (Microsoft)	None	None	MIT, as above

Item	Licence fee	Ongoing (a subscription, or per model)	What it asks of us
DINOv3 image encoder (Meta)	None	None	Someone who can bind the company accepts the DINOv3 License. Research publications acknowledge DINOv3.
BiRefNet background removal	None	None	MIT notice
CuMesh, cubvh, FlexGEMM, utils3d	None	None	MIT notices
Eigen (inside o-voxel)	None	None	MPL 2.0; used unmodified, so nothing more
flash-attn, PyTorch	None	None	BSD notices
CUDA runtime, in the base image	None	None	NVIDIA's CUDA licence allows using and shipping the runtime in containers
nvdiffrast (NVIDIA): texture baking in the export	**Unknown.** NVIDIA does not publish a price; a commercial licence is quoted on request.	Set by NVIDIA, if licensed	Non-commercial use only as it stands (section 2)
RMBG-2.0 (BRIA): **not used**	None, because it is not used	None	Would need a paid commercial licence from BRIA. BiRefNet replaces it.

Getting the weights:

Item	Cost
A Hugging Face account for the company	Free. No paid plan (Pro, Enterprise) is needed.
Access to the gated DINOv3 repository	Free: Meta approves the request at no charge
Downloading the weights (about 18 GB, once per environment and per new lock)	Free from Hugging Face
The Hugging Face token	Free. It is only needed for the lock and the copies (section 3).

In total:

- **Licence fees:** none, except nvdiffrast's, which depends on the decision in section 2:

nvdiffrast option	Fee
1. Legal finds our use allowed	None (Legal's time)
2. Licence it from NVIDIA	NVIDIA's quote
3. Replace it	No fee (engineering time)

- **Ongoing:** no subscription, and no fee per model, for any of the rest.